Air New Zealand
Flight Booking System

Project Phase 1 - Presentation

Christian Cantos - 23188023

Background and Context

- Air New Zealand operates in a highly competitive aviation industry.
- Efficiency, accuracy, and customer satisfaction are critical for success.
- Handles large volumes of:
  - Flight schedules
  - Customer bookings
  - Operational data
- The company heavily relies on digital systems for their daily operations such as booking flights, flight management, and scheduling.
- Currently relies on a disconnected systems that results to:
  - Lack of coordination and integration
  - Operational inefficiencies

Stakeholders

- Customer
  - Search and view flights
  - Book flights,
  - Manage Bookings
- Guests
  - Search and view flights
- Admin
  - Flight scheduling
  - Monitor seat availability
  - Flight management
- Airline Management
  - Relies on system’s output
- System Developers
  - Builds and design the system
- IT Support / System Administrator
  - Maintains & fixes the system

Flight Booking System

- To addressed these challenges, there is a clear need for a centralized, integrated flight booking system that can:
  - Provide a single platform for managing flights, bookings, and customer data.
  - Enable real-time updates and accurate tracking of flight and booking information
  - Automate key processes such as booking, cancellations, and seat management
  - Improve coordination between staff and departments
  - Enhance the overall customer experience through faster and more reliable services.

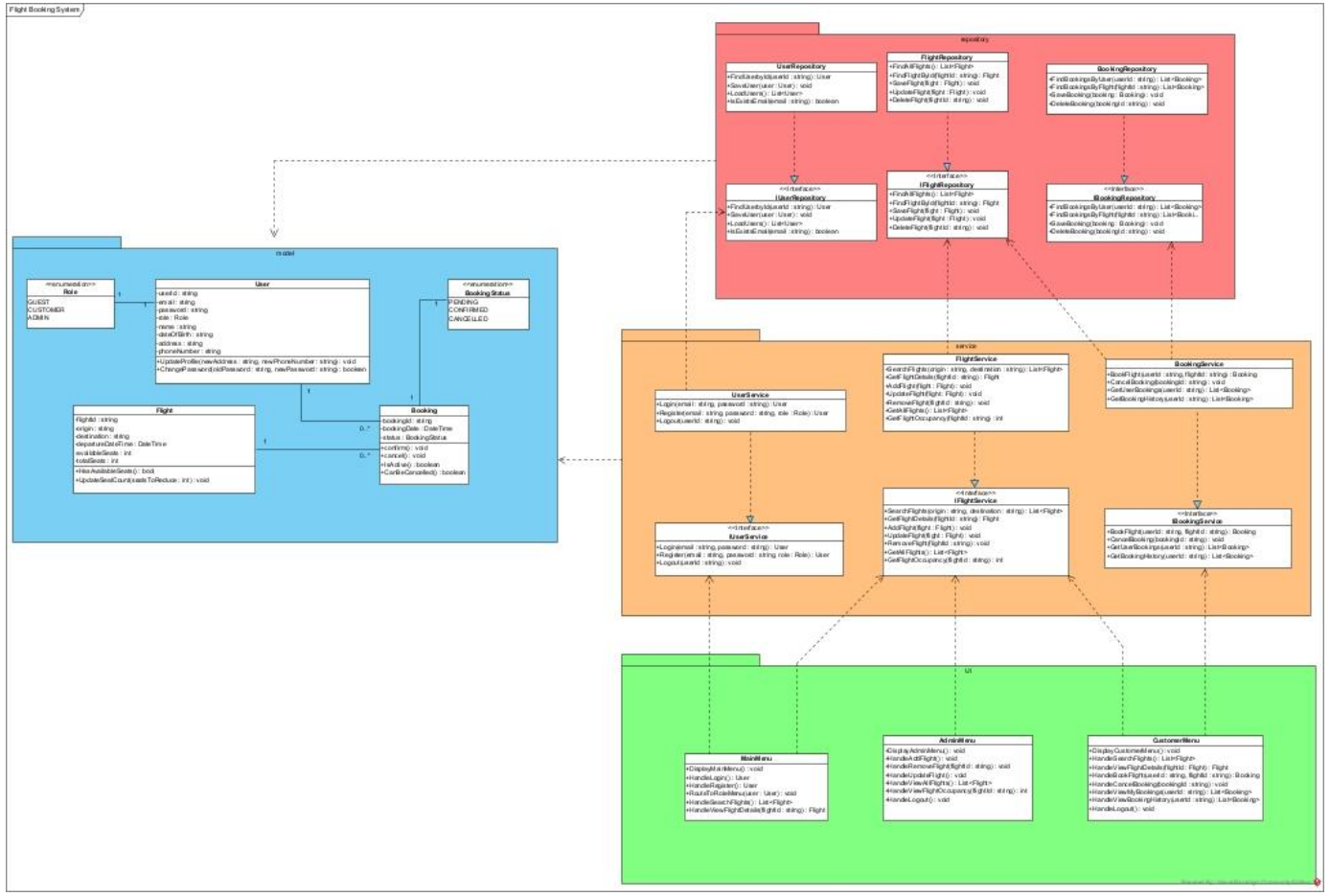
UML Class Diagram

Repository Layer

Model Layer

Service Layer

UI Layer



OOP Principles

Encapsulation

All fields are private. Updating seats are only accessed through method UpdateSeatCount() – this protects the internal state of the flight class and controlling how seats are modified.

Abstraction

Used interface classes, which separates the system's operations and how they are being implemented – It allows the system to be flexible and easier for testing

Polymorphism

Interface-based polymorphism. IFlightService, IBookingService, IUserService can be user for different implementations, and it allows the system to interact with these services. It improves flexibility, maintainability and testability.

Separation of concerns

Dividing the system into 4 layers: model, repository, service and UI – each layer has their own specific responsibilities which improves maintainability and scalability.

High Cohesion

Each class handles specific behaviors, such as Booking class handles Confirm() which changes the booking status, FlightService handles flight operations.

Low Coupling

Minimizes dependencies between classes. Through the use of interface. Components such as MainMenu, AdminMenu, CustomerMenu does not directly uses concrete service classes rather it interacts with services through interfaces – which allows flexibility

Live Demo

- Core Functionalities:
  - User authentication
  - Role-base Access
  - Flight management
  - And booking operations